2023 International Conference on Recent Advances in Electrical, Electronics, Ubiquitous Communication, and Computational Intelligence (RAEEUCCI) | 979-8-3503-3742-6/23/$31.00 ©2023 IEEE | DOI: 10.1109/RAEEUCCI57140.2023.10134313

Detecting Handwritten digits using fully connected Artificial Neural Network in FPGA which will act like a replacement for GPU

Ruchi Sinha
Department of Electronics and Communication Engineering
SRM Institute of Science and Technology
Chennai, India
rs7177@srmist.edu.in

Shubham Kumar Gupta
Department of Electronics and Communication Engineering
SRM Institute of Science and Technology
Chennai, India
sg2749@srmist.edu.in

Dr. Prithiviraj Rajalingam
Department of Electronics and Communication Engineering
SRM Institute of Science and Technology
Chennai, India
prithivr@srmist.edu.in

Dr. Somyaranjan Routray
Department of Electronics and Communication Engineering
SRM Institute of Science and Technology
Chennai, India
soumyars@srmist.edu.in

***Abstract*— AI/ML has grown significantly in the last few years, and slowly and steadily it is starting to impact the semiconductor industry as well. The world is moving towards more application specific technologies. We first saw CPUs, then GPUs came along for image specific applications, now, a new revolution is trying to take place by implementing Neural Networks in FPGA. In this paper, a 5-layer fully connected artificial neural network has been built using the MNIST dataset to recognize handwritten digits. Software like Vivado 2022.1 and in built Zynq 7000 board has been used to realize the implementation. An accuracy of 98% has been achieved through this implementation, and very minimal power and memory usage has also been seen. Regarding the paper, it has been clearly seen that ANN based implementation of FPGA is far superior to the workings of a GPU because of its efficient memory and power usage, also it creates less latency issue as FPGA works in a clock cycle of nanoseconds.**

Keywords*— *VLSI, FPGA, Deep learning, Neural Network, GPU

I. INTRODUCTION

One of the few revelations have been that Neural Network implementation in FPGA acts superior to general GPU use for any sort of image or signal processing. FPGA consumes less power and has lesser latency issue as it deals with clock cycles in nanoseconds. But hardware implementation of neural network comes with its own challenges, even though the implementation of the neural network itself is faster in hardware but the computational complexity can reduce with higher number of weights and neurons, hence in this paper, we have tried to resolve that issue by using fixed point representation of number and using signed multiplication and addition of numbers. With the limitations that come with an FPGA implementation of ANN, higher accuracy is also harder to achieve but through the small variations suggested, an industry level accuracy can be gained. Also, as no IP cores have been used, this makes the design of the neural network more flexible and scalable. Our aim was to build such an implementation that it can meet industry requirements and can have flexible usage with different number of inputs and datasets. On top of that, an effort was made to make it more memory and power efficient for users.

II. ACRONYMS

- AI – Artificial Intelligence
- ANN- Artificial Neural Network
- CPU- Central Processing Unit
- FPGA- Field Programmable Gate Array
- GPU- Graphic Processing Unit
- ML- Machine Learning

III. LITERATURE SURVEY

The proposed implementation of a fully connected Artificial Neural Network (ANN) in FPGA was inspired by previous research in the field of CNN-based FPGA implementations. The paper on Binary weighted convolutional artificial neural network [1] served as a trigger point for our study, motivating us to explore a fully connected ANN in FPGA for our specific application of handwritten digit recognition.

Previous studies have utilized various forms of CNN in FPGA, including custom data and hardware platforms such as IVS-70 [2] and XILINX Virtex-5 [3] FPGA, for tasks such as object detection. In our implementation, we are using the Zynq 7000 board for realizing handwritten digit recognition.

We also observed that fingertip digit recognition using CNN in FPGA has been realized in previous research [4], but our implementation with ANN has achieved similar accuracy. Additionally, our project architecture is more dynamic and utilizes Block RAMs (BRAMs) instead of Static RAMs (SRAMs) as used in some other studies [5]. Moreover, our architecture is sequential and scalable, providing flexibility for future improvements.

Overall, our proposed implementation of ANN in FPGA for handwritten digit recognition builds upon previous research in the field, leveraging the advantages of a fully connected ANN architecture and the scalability and dynamic nature of FPGA-based implementations, while achieving comparable accuracy to other studies using CNN in FPGA.

IV. METHODOLOGY

The most basic and essential element of a neural network is a neuron. So, at first build a neuron, for a neuron, we have a weight function, which is pre-trained, an activation function, in our case we used Sigmoid and ReLU.

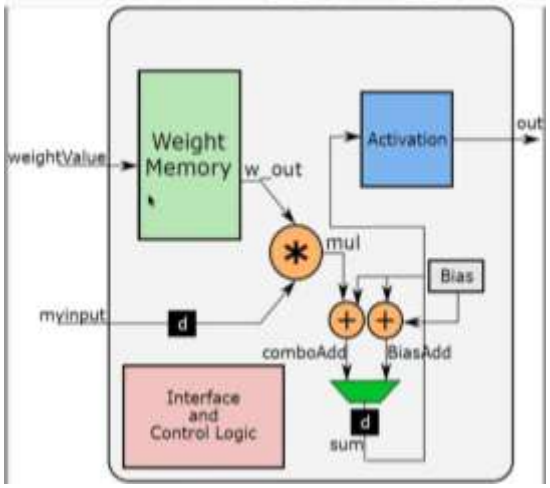

Fig 1 – Block Diagram of neuron

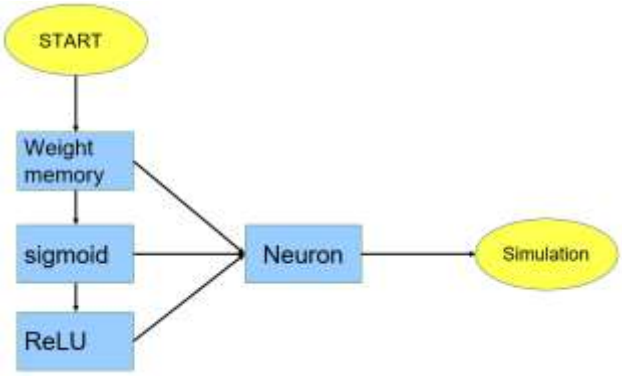

Fig 2 - Flow chart of Neuron

In a neuron as shown in figure 1, there is an input, a weight, and a bias. Mathematically, a neuron is basically: input.weight + bias.

The flow chart given in figure 2 describes the architecture while writing the code in Verilog. The top layer is neuron, which is made up of weight memory and activation function (Sigmoid/ReLU).

As this paper aims at working with HDL, the input, weight and bias are all binary files, and are generated using python codes.

The output we get after this simple dot product and addition is then given to an activation function. In our code, we used two very famous activation functions mainly ReLU and Sigmoid.

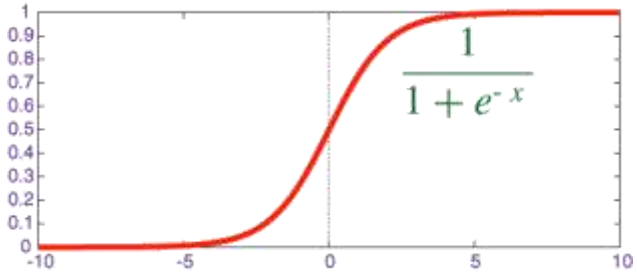

Fig 3- Sigmoid function

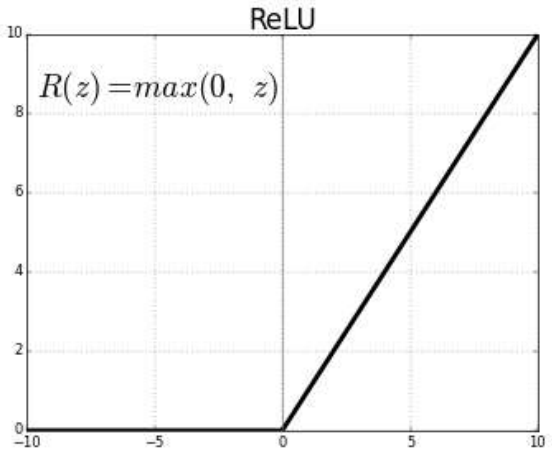

Fig 4- ReLU function

Figure 3 demonstrates that the sigmoid function permits negative values, as opposed to figure 4 where ReLU restricts outputs to zero for all negative inputs. In ReLU, any negative value is truncated to zero, whereas sigmoid employs an exponential function to determine the output. In instances of output overflow, the neuron is mapped to the highest value it can generate, while in cases of underflow, the neuron is mapped to its lowest possible value.

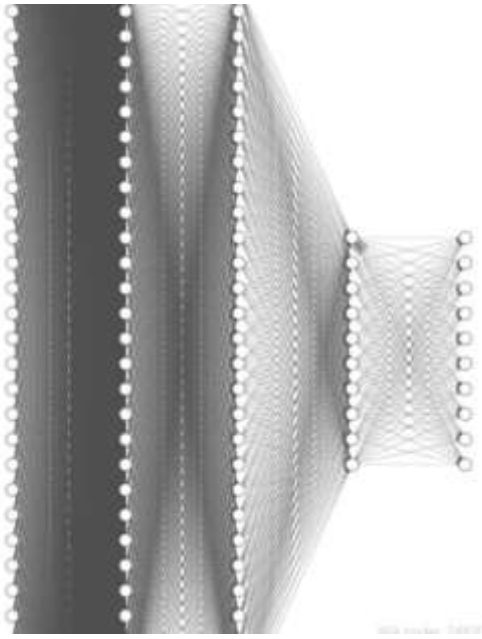

Fig 5 – The proposed architecture of Artificial Neural Network built

The neural network construction in the second stage comprises a proposed architecture, as depicted in figure 5, encompassing a total of 5 layers. These layers include one input layer with 784 inputs, corresponding to the dimensions of 28x28. Additionally, there are 3 hidden layers, with the first and second hidden layers each containing 30 neurons, and the third hidden layer comprising 10 neurons. Lastly, the output layer encompasses 10 neurons. Notably, a specialized function was incorporated to determine the maximum number of weights required for detecting the desired number. This neural network is characterized by being fully connected and feed-forward in nature, with no utilization of backward propagation algorithm.

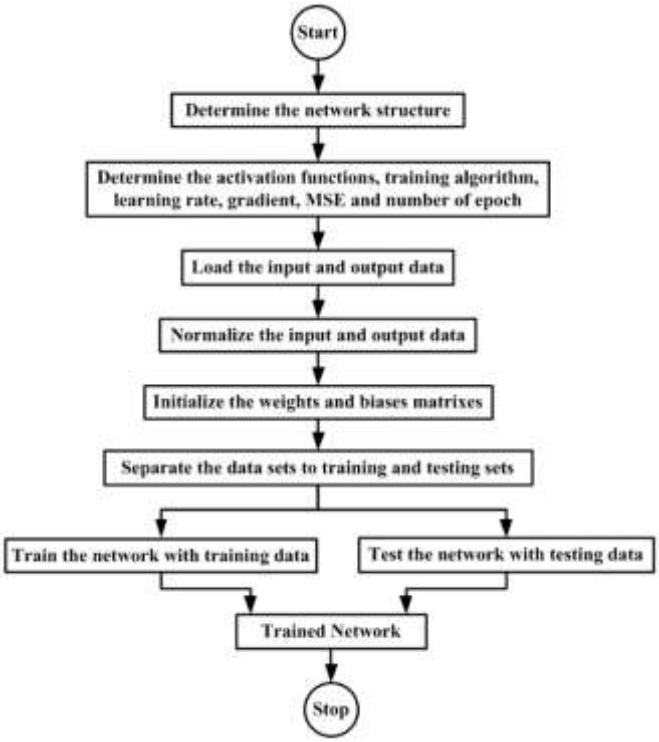

Fig 6- Step by step process of building an ANN

The training process to establish the necessary weights and biases was executed in Python, as depicted in figure 6, with the resulting files saved in binary format.

The automation of the weights and biases generation contributed to the attainment of a high accuracy rate. The layers of the neural network were constructed using Verilog, and the binary files containing the weights and biases were read by the neurons during the inference phase. It's worth noting that the MNIST database, utilized for training and testing, was also in the form of binary text files, as HDL languages are designed to read data in binary or hexadecimal formats. The input layer is not designed as it just passes the input, we are already giving the inputs through the text MNIST database file.

The remaining four layers of the neural network comprise two layers with 30 neurons each, and two additional layers with 10 neurons each. The architecture of the neural network is highly customizable, facilitated by Python codes that can be adjusted to accommodate varying inputs, weights, and biases. Additionally, a specialized function has been devised to analyze the maximum weights across the layers and accurately detect the desired number. This flexibility in design and the incorporation of specialized functions enhance the versatility and adaptability of the neural network for various applications.

A test bench is also written to calculate the accuracy and show the output in the console, the test bench again includes all the files, layers and MNIST database inputs.

V. Results and Discussions

During the implementation of the neuron, it was observed that sigmoid function performed significantly better in the chosen architecture and hardware specifications. Sigmoid utilized 99% of dynamic power and required a smaller number of flip-flops and look-up tables (LUTs) compared to ReLU, which consumed 93% of dynamic power.

As a result, sigmoid was chosen for the implementation of the neural network. The accuracy achieved with sigmoid was 98%, while ReLU yielded an accuracy of 91%. This indicates that sigmoid was more effective in achieving higher accuracy for the specific implementation and hardware setup, underscoring the importance of selecting the appropriate activation function for optimal performance in a given context.

Table 1 – Comparison Table between ReLU and Sigmoid

Parameters	ReLU	Sigmoid
Flipflops	66	52
LUTs	58	47
Capacity of neurons that can be used	706	872
Dynamic Power used	93%	99%
Accuracy	91%	98%

In the third stage, we utilized inbuild GPUs in TensorFlow to implement a neural network. However, the accuracy achieved was 97.64%, which was lower than our model's accuracy. Furthermore, we observed that TensorFlow allocated the entire memory of the GPU for the neural network, which contrasted with the FPGA-based implementation where memory allocation was not an issue. It is also compared the number of look-up tables (LUTs) and flip-flops used in our FPGA-based implementation with the size of the GPU, and determined that FPGA was superior in terms of space utilization, memory efficiency, and power consumption. This underscores the advantages of FPGA-based implementations in terms of resource utilization and efficiency compared to GPU-based implementations for our specific application.

Table 2- Parameters of the Neural Network

Parameters	Neural Network
LUT	6370
Flipflops	5369
BRAM	15
DSP	160
IO	105
BUFG	1
Dynamic Power	99 %

did observe a significant rise in temperature during the implementation, which could be an area of improvement for future studies. If the temperature rise can be minimized while maintaining the same level of accuracy, the implementation would be more successful. This highlights the need for-

Table 3- Comparison with Reviewed sources

Parameters	[1]	[2]	[3]	[4]	[5]	Proposed System
Data inputs for training	-	60000	-	50000	-	10000
Accuracy	-	-	-	-	97 %	98 %
RAMs used	DRAMs	-	-	-	SRAMs	BRAMs
FPGA used	-	-	Virtex-5	-	-	Zynq 7000

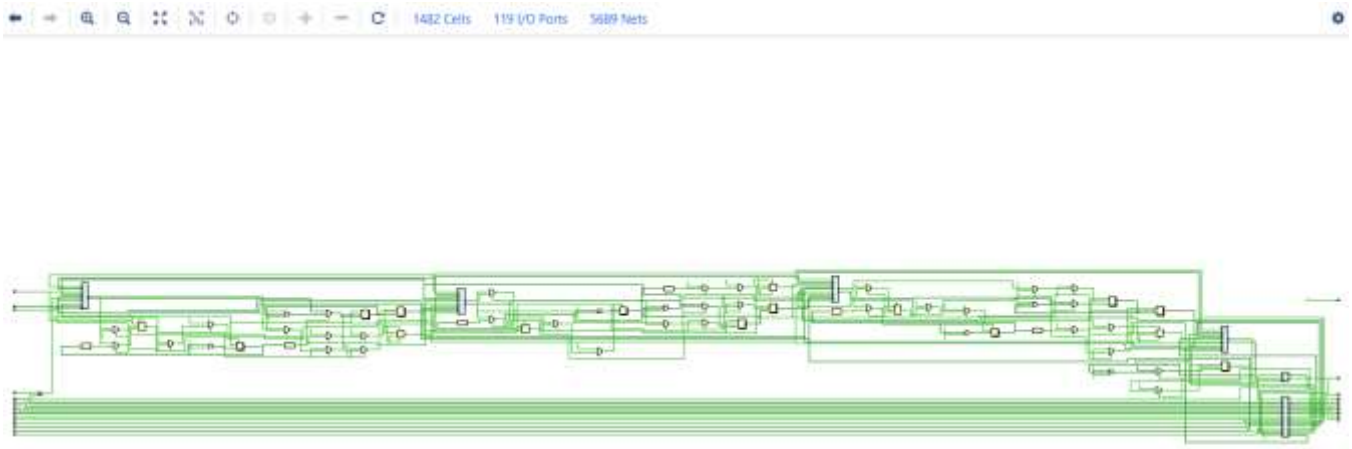

Fig 7- RTL Schematic of Neural Network

VI. CONCLUSION

The implementation of Artificial Neural Network (ANN) in FPGA, as proposed, exhibited higher accuracy compared to previous studies on Convolutional Neural Networks (CNN) in FPGA. This can be attributed to the fact that ANN is feed-forward and fully connected, requiring fewer data inputs but achieving high accuracy. In our proposed model, we used 10,000 data inputs, whereas other studies have used higher numbers, ranging from 20,000 to 60,000 data inputs. ANN is known for its ability to effectively detect relationships between dependent and independent variables.

Furthermore, the proposed ANN model outperformed the GPU implementation in terms of accuracy. However, we careful consideration of thermal management strategies in FPGA-based implementations to ensure reliable and efficient operation in high-performance computing applications. Nonetheless, the accuracy achieved in the proposed model was higher than our expectations, indicating the potential of ANN in FPGA-based implementations for achieving high accuracy in machine learning tasks

REFERENCES

[1] Charles Rajesh Kumar J, Vinod Kumar D, Baskar D, Mary Arusni B, Jenova R., "VLSI design and implementation of High-performance Binary-weighted convolutional artificial neural networks for embedded vision based Internet of Things (IoT)," in 16th International Learning & Technology conference 2019

[2] Daniel Jonasson, "Object Recognition Through Convolutional Learning For FPGA," in M¨alardalen University School of Innovation Design and Engineering V¨aster˚as, Sweden, 2017

[3] Balwinder Singh, Robin Khosla, Shashi Bala Implementation of Efficient Object Detection Algorithm on FPGA, IJECT Vol. 6

[4] MD Shahbaz Khan, Niharika, Priya Yadav, Rishabh Verma, Dr Indu Sreedevi, "FPGA Simulation of Fingertip Digit Recognition Using CNN," in 2020 7th International Conference on Signal Processing and Integrated Networks (SPIN)

[5] A Mingu Kang, Sungmin, Sujan Gonugondla, Naresh R. Shanbhaj, "An In-Memory VLSI Architecture for Convolutional Neural Networks, Mingu Kang, Sungmin Lim, Sujan Gonugondla, and Naresh R. Shanbhag," IEEE Journal on Emerging and selected topics in circuits and system, sep 2018

[6] Gaber, Lamya, Aziza I. Hussein, and Mohammed Moness. "Fault Detection based on Deep Learning for Digital VLSI Circuits." *Procedia Computer Science* 194 (2021): 122-131.

[7] Zhao, Zhong-Qiu, Peng Zheng, Shou-tao Xu, and Xindong Wu. "Object detection with deep learning: A review." *IEEE transactions on neural networks and learning systems* 30, no. 11 (2019): 3212-3232.

[8] Ardakani, Arash, François Leduc-Primeau, Naoya Onizawa, Takahiro Hanyu, and Warren J. Gross. "VLSI implementation of deep neural network using integral stochastic computing." *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 25, no. 10 (2017): 2688-2699.

[9] Lin, Shinfeng D., Tingyu Chang, and Wensheng Chen. "Multiple Object Tracking using YOLO-based Detector." *Journal of Imaging Science and Technology* 65, no. 4 (2021): 40401-1.

[10] Korekado, Keisuke, Takashi Morie, Osamu Nomura, Hiroshi Ando, Teppei Nakano, Masakazu Matsugu, and Atsushi Iwata. "A VLSI convolutional neural network for image recognition using merged/mixed analog-digital architecture." *Journal of Intelligent & Fuzzy Systems* 15, no. 3-4 (2004): 173-179.